

Experiment No. 4

Name: Soham Satpute

Batch: B

Div: D20A

Roll No: 53

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- **Function Visibility** defines who can access a function:

- o public: available both inside and outside the contract.
- o private: only accessible within the same contract.
- o internal: accessible within the contract and its child contracts.
- o external: can be called only by external accounts or other contract
- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays**: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings**: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs**: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums**: Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

- **storage**: Data stored permanently on the blockchain. Examples: state variables.
- **memory**: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata**: A non-modifiable and non-persistent location used for external function

parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

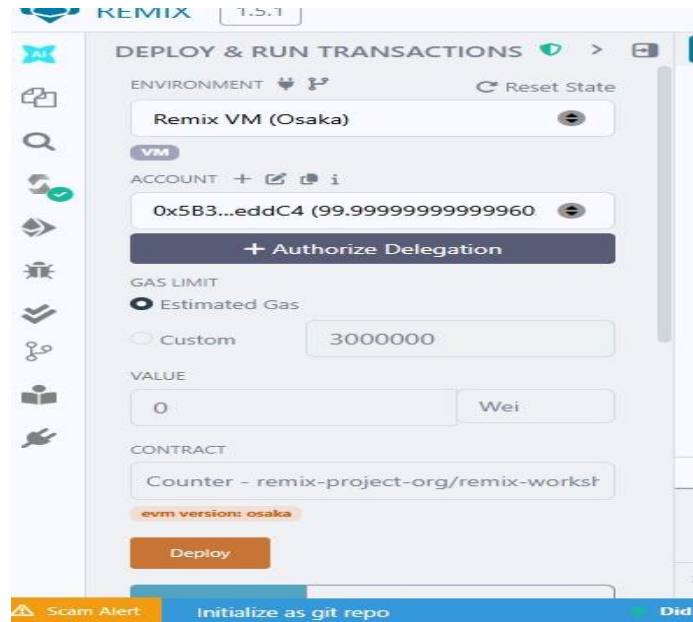
- **Ether and Wei:** Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like `transfer()` and `send()` are commonly used, while `call()` provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

Implementation:

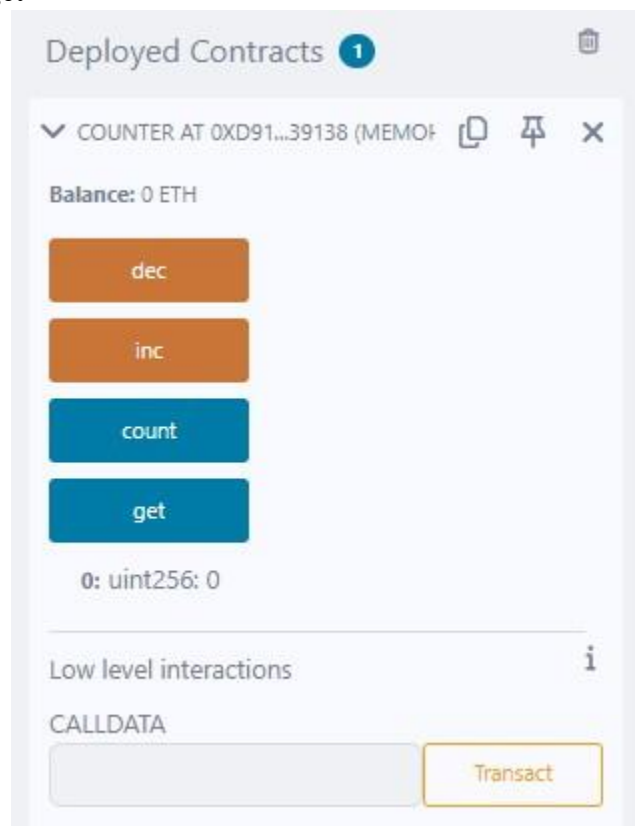
- Tutorial no. 1 – Compile the code

```
1  // SPDX-License-Identifier: MIT
2  //Soham Satpute
3  pragma solidity ^0.8.3;
4
5  contract Counter {
6      uint public count;
7
8      // Function to get the current count
9      function get() public view returns (uint) { 2453 gas
10         return count;
11     }
12
13     // Function to increment count by 1
14     function inc() public { infinite gas
15         count += 1;
16     }
17
18     // Function to decrement count by 1
19     function dec() public { infinite gas
20         count -= 1;
21     }
22 }
```

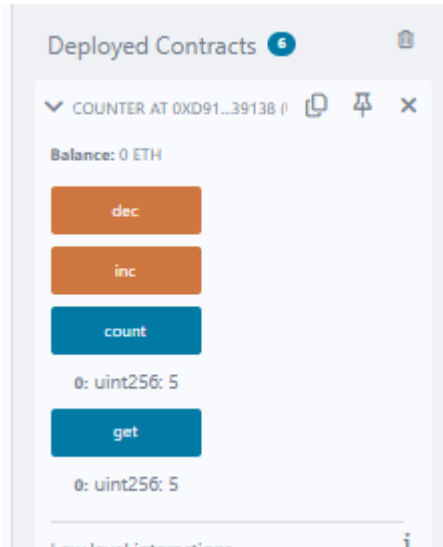
1 – Deploy the contract



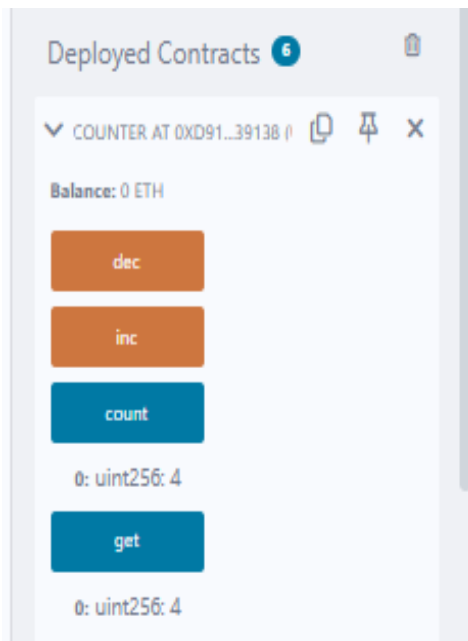
- Tutorial no. 1 – get



- Tutorial no.
- Tutorial no. 1 – Increment



- Tutorial no. 1 – Decrement



- Tutorial no.2:

REMIX 1.5.1

learneth tutorials

LEARNETH

Tutorials list

Syllabus

2. Basic Syntax 2 / 19

Using the `pragma` keyword (line 3), we specify the Solidity version we want the compiler to use. In this case, it should be greater than or equal to `0.8.3` but less than `0.9.0`.

We define a contract with the keyword `contract` and give it a name, in this case, `HelloWorld` (line 5).

Inside our contract, we define a `state variable` `greet` that holds the string `"Hello World"` (line 6).

Solidity is a *statically typed* language, which means that you need to specify the type of the variable when you declare it. In this case, `greet` is a `string`.

We also define the *visibility* of the variable, which specifies from where you can access it. In this case, it's a `public` variable that you can access from inside and outside the contract.

Don't worry if you didn't understand some concepts like *visibility*, *data types*, or *state variables*. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.

[Watch a video tutorial on Basic Syntax.](#)

★ Assignment

- Delete the HelloWorld contract and its content.
- Create a new contract named "MyContract".
- The contract should have a public state variable called "name" of the type string.
- Assign the value "Alice" to your new variable.

Check Answer Show answer

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity >=0.8.3 <0.9.0;
3 //Soham Satpute
4 contract MyContract {
5     string public name = "Alice";
6 }
7

```

Explain contract

Listen on all transactions

call to Counter.count

call [call] from: 0x58380a6a781c568545dcfc803fc8b75f56beddC4 to: Counter.count() data: 0x

call to Counter.get

call [call] from: 0x58380a6a781c568545dcfc803fc8b75f56beddC4 to: Counter.get() data: 0x6c

- Tutorial no. 3

Tutorials list

Syllabus

3. Primitive Data Types 3 / 19

int

We use the keywords `int` and `uint` to declare an Integer type. Integers can be positive, negative, or zero and range from 8 bits to 256 bits. The type `int` is the same as `int256`.

address

Variables of the type `address` hold a 20-byte value, which is the size of an Ethereum address. There is also a special kind of Ethereum address, `address payable`, which can receive ether from the contract.

All these data types have default values, as shown in the contract (line 29).

You can learn more about these data types as well as *Fixed Point Numbers*, *Byte Arrays*, *Strings*, and more in the [Solidity documentation](#).

Later in the course, we will look at data structures like **Mappings**, **Arrays**, **Enums**, and **Structs**.

[Watch a video tutorial on Primitive Data Types.](#)

★ Assignment

- Create a new variable `newAddr` that is a `public` `address` and give it a value that is not the same as the available variable `addr`.
- Create a `public` variable called `neg` that is a negative number, decide upon the type.
- Create a new variable, `newU` that has the smallest `uint` size type and the smallest `uint` value and is `public`.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

Check Answer Show answer

Next

Well done! No errors.

```

15 uint8 public u8 = 1;
16 uint public u256 = 456;
17 uint public u = 123; // uint is an alias for uint256
18
19 /*
20 Negative numbers are allowed for int types.
21 Like uint, different ranges are available from int8 to int256
22 */
23 int8 public i8 = -1;
24 int public i256 = 456;
25 int public i = -123; // int is same as int256
26
27 //Soham Satpute
28 address public addr = 0xCA05b7d915458f5480de686bdfc3f44e9fa732c;
29 address public newAddr = 0x7a250d5630BACf539739dF2C590Cb4c659F2488D; // Uniswap V2 Router address
30 int8 public neg = -123;
31 uint8 public newU = 0;
32
33 // Default values
34 bool public defaultBool; // false
35 uint public defaultUint; // 0
36 int public defaultInt; // 0
37

```

Explain contract

Listen on all transactions

Filter with transaction hash or address

call [call] from: 0x58380a6a781c568545dcfc803fc8b75f56beddC4 to: Counter.count() data: 0x066...61abd

call to Counter.count

call [call] from: 0x58380a6a781c568545dcfc803fc8b75f56beddC4 to: Counter.count() data: 0x066...61abd

call to Counter.get

call [call] from: 0x58380a6a781c568545dcfc803fc8b75f56beddC4 to: Counter.get() data: 0x6d4...ce63c

Debug

Debug

Debug

RemixAI Copilot (experimental)

- Tutorial no.4

Tutorials list

Syllabus

4. Variables

4 / 19

2. Local Variables

Local Variables are stored in the *memory* and their values are only accessible within the function they are defined in. Local Variables are not stored on the blockchain. In this contract, the `uint i` (line 11) is a local variable.

3. Global Variables

Global Variables, also called *Special Variables*, exist in the global namespace. They don't need to be declared but can be accessed from within your contract. Global Variables are used to retrieve information about the blockchain, particular addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

- Create a new public state variable called `blockNumber`.
- Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check AnswerShow answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     //Sohan Satpute
7     string public text = "Hello";
8     uint public num = 123;
9     uint256 public blockNumber;
10
11     function doSomething() public { 22334 gas
12         // Local variable (not stored on blockchain)
13         uint i = 456;
14
15         // Global variables
16         uint timestamp = block.timestamp;
17         address sender = msg.sender;
18
19         // Store current block number in state variable
20         blockNumber = block.number;
21     }
22 }
```

AI Explain contract

0 Listen on all transactions Filter with transaction hash or ad...

[call] from: 0x58380a6a781c568545dcfc803fc8875f56bedd04 to: Counter.count() data: 0x066...61abd
call to Counter.count

[call] from: 0x58380a6a781c568545dcfc803fc8875f56bedd04 to: Counter.count() data: 0x066...61abd
call to Counter.get

[call] from: 0x58380a6a781c568545dcfc803fc8875f56bedd04 to: Counter.get() data: 0xd04...ce63c
call to Counter.get

- Tutorial no. 5

Tutorials list

Syllabus

5.1 Functions - Reading and Writing to a State Variable

5 / 19

Solidity functions can be split into two types:

- Functions that modify the state of the blockchain, like writing to a state variable. In this contract, the `set` function (line 9) changes the state variable `num`.
- Functions that don't modify the state of the blockchain. These functions are marked `view` or `pure`. For example, in this contract, the `get` function (line 14) marked `view` that only returns `num` does not change the state.

To define a function, use the `function` keyword followed by a unique name.

If the function takes inputs like our `set` function (line 9), you must specify the parameter types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `set` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

[Watch a video tutorial on Functions.](#)

★ Assignment

- Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
- Create a public function called `get_b` that returns the value of `b`.

Check AnswerShow answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7     bool public b = true;
8
9     // You need to send a transaction to write to a state variable.
10    function set(uint _num) public { 22536 gas
11        num = _num;
12    }
13
14    // You can read from a state variable without sending a transaction.
15    function get() public view returns (uint) { 2475 gas
16        return num;
17    }
18    function get_b() public view returns (bool) { 2539 gas
19        return b;
20    }
21 }
```

AI Explain contract

0 Listen on all transactions Filter with transaction hash or ad...

[call] from: 0x58380a6a781c568545dcfc803fc8875f56bedd04 to: Counter.count() data: 0x066...61abd
call to Counter.count

[call] from: 0x58380a6a781c568545dcfc803fc8875f56bedd04 to: Counter.count() data: 0x066...61abd
call to Counter.get

[call] from: 0x58380a6a781c568545dcfc803fc8875f56bedd04 to: Counter.get() data: 0xd04...ce63c
call to Counter.get

- Tutorial no.6:

Tutorials list

5.2 Functions - View and Pure

6 / 19

"In addition to the list of state modifying statements explained above, the following are considered reading from the state:

- Reading from state variables.
- Accessing `address(this).balance` or `address(address).balance`.
- Accessing any of the members of block, tx, msg (with the exception of `msg.sig` and `msg.data`).
- Calling any function not marked pure.
- Using inline assembly that contains certain opcodes."

From the [Solidity documentation](#).

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions `view` and `pure` can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

[Watch a video tutorial on View and Pure Functions.](#)

★ **Assignment**

Create a function called `addTx2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer

Show answer

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addTx(uint y) public view returns (uint) {
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) {
14        return i + j;
15    }
16
17    function addTx2(uint256 y) public {
18        x += y;
19    }

```

Explain contract

Listen on all transactions

Filter with transaction hash or address

[call] from: 0x58380a6a701c568545dcfc803fc8875f56bedd04 to: Counter.count() data: 0x066...61abd

call to Counter.count

Debug

[call] from: 0x58380a6a701c568545dcfc803fc8875f56bedd04 to: Counter.count() data: 0x066...61abd

call to Counter.get

Debug

[call] from: 0x58380a6a701c568545dcfc803fc8875f56bedd04 to: Counter.get() data: 0x6d4...ce63c

Debug

- Tutorial no. 7

REMX 1.5.1

LEARNETH

Tutorials list

5.3 Functions - Modifiers and Constructors

7 / 19

error.

You can learn more about `assert` and `require` in the [Solidity documentation](#), they are used to check for conditions and throw errors if they are not met.

The `solidAddress` modifier (line 28) has a parameter of type `address` and checks if the provided address is valid. If it is, it continues to execute the code.

Constructor

A constructor function is executed upon the creation of a contract. You can use it to run contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

[Watch a video tutorial on Function Modifiers.](#)

★ **Assignment**

- Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
- Make sure that `x` can only be increased.
- The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer

Show answer

Next

Well done! No errors.

```

40
41 modifier noReentrancy() {
42     require(!locked, "No reentrancy");
43     locked = true;
44     _;
45     locked = false;
46 }
47
48 //Soham Satpute
49 function decrement(uint i) public noReentrancy {
50     x -= i;
51
52     if (i > 1) {
53         decrement(i - 1);
54     }
55
56     function increaseX(uint256 amount) public {
57         unchecked {
58             x += amount;
59         }
60     }

```

Explain contract

Listen on all transactions

Filter with transaction hash or address

[call] from: 0x58380a6a701c568545dcfc803fc8875f56bedd04 to: Counter.count() data: 0x066...61abd

call to Counter.get

Debug

[call] from: 0x58380a6a701c568545dcfc803fc8875f56bedd04 to: Counter.get() data: 0x6d4...ce63c

Debug

- Tutorial no.8

LEARNETH

Tutorials list

Syllabus

5.4 Functions - Inputs and Outputs
8 / 19

you can omit (leave out) the return statement and return them individually.

Deconstructing Assignments

You can use deconstructing assignments to unpack values into distinct variables.

The `deconstructingAssignments` function (line 49) assigns the values of the `returnMany` function to the new local variables `i`, `b`, and `j` (line 60).

Input and Output restrictions

There are a few restrictions and best practices for the input and output parameters of contract functions.

"[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible."
From the Solidity documentation.

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on Function Outputs.

★ Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer Show answer

Next

Well done! No errors.

```
67 // Cannot use map for neither input nor output
68
69
70 // Can use array for input
71 function arrayInput(uint[] memory _arr) public {} infinite gas
72
73 // Can use array for output
74 //Soham Satpute
75 uint[] public arr;
76
77 function arrayOutput() public view returns (uint[] memory) { infinite gas
78     return arr;
79 }
80
81 function returnTwo() public pure returns (int, bool) { 496 gas
82     int x = -2;
83     bool y = true;
84     return (x, y);
85 }
86
87 }
```

Explain contract

0 Listen on all transactions

call to Counter.count

[call] from: 0x5B38Da6a701c568545dCfcB03FcB875F56beddC4 to: Counter.count() data: 0x066...61ab

call to Counter.get

[call] from: 0x5B38Da6a701c568545dCfcB03FcB875F56beddC4 to: Counter.get() data: 0x6d4...ce63c

- Tutorial no. 9

LEARNETH

Tutorials list

Syllabus

5.4 Functions - Inputs and Outputs
8 / 19

you can omit (leave out) the return statement and return them individually.

Deconstructing Assignments

You can use deconstructing assignments to unpack values into distinct variables.

The `deconstructingAssignments` function (line 49) assigns the values of the `returnMany` function to the new local variables `i`, `b`, and `j` (line 60).

Input and Output restrictions

There are a few restrictions and best practices for the input and output parameters of contract functions.

"[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible."
From the Solidity documentation.

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on Function Outputs.

★ Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer Show answer

Next

Well done! No errors.

```
60 (uint i, bool b, uint j) = returnMany();
61
62 // Values can be left out.
63 (uint x, , uint y) = (4, 5, 6);
64
65 return (i, b, j, x, y);
66 }
67
68 // Cannot use map for neither input nor output
69
70 // Can use array for input
71 function arrayInput(uint[] memory _arr) public {} infinite gas
72
73 // Can use array for output
74 //Soham Satpute
75 uint[] public arr;
76
77 function arrayOutput() public view returns (uint[] memory) { infinite gas
78     return arr;
79 }
80
81 function returnTwo() public pure returns (int, bool) { 496 gas
82     int x = -2;
83     bool y = true;
84 }
```

Explain contract

0 Listen on all transactions

call to Counter.count

[call] from: 0x5B38Da6a701c568545dCfcB03FcB875F56beddC4 to: Counter.count() data: 0x066...61abd

call to Counter.get

[call] from: 0x5B38Da6a701c568545dCfcB03FcB875F56beddC4 to: Counter.get() data: 0x6d4...ce63c

• Tutorial no.10

LEARNETH

Tutorials list

7.1 Control Flow - If/Else

10/19

If the statement is true, the function returns `1`.

else

The `else` statement enables our contract to perform an action if conditions are not met. In this contract, the `foo` function uses the `else` statement (line 10) to return `2` if none of the other conditions are met.

else if

With the `else if` statement we can combine several conditions. If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `3`.

Watch a video tutorial on the [If/Else statement](#).

★ **Assignment**

Create a new function called `evenCheck` in the `ifElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer

Show answer

Next

Well done! No errors.

Compile

visibility.sol

ifElse.sol

ifElse_answer.sol

```

8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29

```

//SOHAM SATPUTE

```

function ternary(uint _x) public pure returns (uint) {
    // if (_x < 10) {
    //     return 1;
    // }
    // return 2;

    // shorthand way to write if / else statement
    return _x < 10 ? 1 : 2;
}

function evenCheck(uint y) public pure returns (bool) {
    return y%2 == 0 ? true : false;
}

```

AI copilot

0

Listen on all transactions

Filter with transaction hash or ad...

transaction to Counter.dec pending ...

✓

[vm] from: 0x583...eddC4 to: Counter.dec() 0xd8b...33fa8 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0x18e...5c0b2

call to Counter.count

Debug

call

[call] from: 0x58380a6a701c568545dCfcB03FcB875f56beddC4 to: Counter.count() data: 0x066...61abd

call to Counter.get

Debug

• Tutorial no. 11

LEARNETH

Tutorials list

7.2 Control Flow - Loops

11/19

of iterations that can occur.

do while

The `do while` loop is a special kind of while loop where you can ensure the code is executed at least once, before checking on the condition.

continue

The `continue` statement is used to skip the remaining code block and start the next iteration of the loop. In this contract, the `continue` statement (line 10) will prevent the second if statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on [Loop statements](#).

★ **Assignment**

- Create a public `uint` state variable called `count` in the `Loop` contract.
- At the end of the for loop, increment the count variable by 1.
- Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer

Show answer

Next

Well done! No errors.

Compile

visibility.sol

ifElse.sol

ifElse_answer.sol

loops.sol

loops_answer.sol

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22

```

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.3;

//SOHAM SATPUTE

```

contract Loop {
    uint public count;
    function loop() public {
        // for loop
        for (uint i = 0; i < 10; i++) {
            if (i == 5) {
                // Skip to next iteration with continue
                continue;
            }
            if (i == 5) {
                // Exit loop with break
                break;
            }
            count++;
        }
    }
}

```

AI copilot

0

Listen on all transactions

Filter with transaction hash or ad...

transaction to Counter.dec pending ...

✓

[vm] from: 0x583...eddC4 to: Counter.dec() 0xd8b...33fa8 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0x18e...5c0b2

call to Counter.count

Debug

call

[call] from: 0x58380a6a701c568545dCfcB03FcB875f56beddC4 to: Counter.count() data: 0x066...61abd

call to Counter.get

Debug

- Tutorial no,12

LEARNETH 1.5.1

learneth tutorials

Compile

arraysAndConstructors.sol inputsAndOutputs.sol visibility.sol ifElse.sol loops

Tutorials list **Syllabus**

8.1 Data Structures - Arrays 12 / 19

our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer

Show answer

Next

Well done! No errors.

```

44 }
45 }
46
47 contract CompactArray {
48     uint[] public arr;
49     // SOHAM SATPUTE
50     // Deleting an element creates a gap in the array.
51     // One trick to keep the array compact is to
52     // move the last element into the place to delete.
53     function remove(uint index) public {
54         // Move the last element into the place to delete
55         arr[index] = arr[arr.length - 1];
56         // Remove the last element
57         arr.pop();
58     }
59
60     function test() public {
61         arr.push(1);
62         arr.push(2);
63         arr.push(3);
64         arr.push(4);
65         // [1, 2, 3, 4]
66
67         remove(1);
68         // [1, 4, 3]
69
70         remove(2);
71         // [1, 4]
72     }
73 }

```

- Tutorial no. 13 :

LEARNETH

Tutorials list **Syllabus**

8.2 Data Structures - Mappings 13 / 19

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer

Show answer

Next

Well done! No errors.

```

20 // Reset the value to the default value.
21 delete balances[_addr];
22 }
23
24
25 contract NestedMapping {
26     // Nested mapping (mapping from address to another mapping)
27     mapping(address => mapping(uint => bool)) public nested;
28
29     function get(address _addr1, uint _i) public view returns (bool) {
30         // You can get values from a nested mapping
31         // even when it is not initialized
32         return nested[_addr1][_i];
33     }
34     // SOHAM SATPUTE
35     function set(
36         address _addr1,
37         uint _i,
38         bool _boo
39     ) public {
40         nested[_addr1][_i] = _boo;
41     }
42
43     function remove(address _addr1, uint _i) public {
44         delete nested[_addr1][_i];
45     }
46 }

```

- Tutorial no.14

REMIX1.5.1learneth tutorials

LEARNETH

Tutorials listSyllabus

8.3 Data Structures - Structs14 / 19

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a structs' member we also use the dot operator and assign it a new value (lines 39 and 45).

Watch a video tutorial on Structs.

★ Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer

Show answer

Next

Well done! No errors.

Compile

ings.solstructs.solarrays_answer.solmappings_answer.solstructs_answer.so

```
24 // todo.completed initialized to false
25
26 todos.push(todo);
27 }
28
29 // Solidity automatically created a getter for 'todos' so
30 // you don't actually need this function.
31 function get(uint _index) public view returns (string memory text, bool completed) {
32     Todo storage todo = todos[_index];
33     return (todo.text, todo.completed);
34 }
35
36 // update text
37 function update(uint _index, string memory _text) public {
38     Todo storage todo = todos[_index];
39     todo.text = _text;
40 }
41 // SOHAM SATPUTE
42 // update completed
43 function toggleCompleted(uint _index) public {
44     Todo storage todo = todos[_index];
45     todo.completed = !todo.completed;
46 }
47
48 function remove(uint _index) public {
49     delete todos[_index];
50 }
51 }
```

Explain contract

AI copilot

- Tutorial no. 15

LEARNETH

Tutorials list

Syllabus

8.4 Data Structures - Enums

15 / 19

Even though enum members are named when you define them, they are stored as unsigned integers, not strings. They are numbered in the order that they were defined, the first member starting at 0. The initial value of status, in this case, is 0.

Accessing an enum value

To access the enum value of a variable, we simply need to provide the name of the variable that is storing the value (line 25).

Updating an enum value

We can update the enum value of a variable by assigning it the `uint` representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

[Watch a video tutorial on Enums.](#)

★ Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

enums.sol

enums_answer.sol

```

28
29 function getSize() public view returns (Size) {
30     return sizes;
31 }
32
33 // Update status by passing uint into input
34 //Soham Satpute
35 function set(Status _status) public {
36     status = _status;
37 }
38
39 // You can update to a specific enum like this
40 function cancel() public {
41     status = Status.Canceled;
42 }
43
44 // delete resets the enum to its first value, 0
45 function reset() public {
46     delete status;
47 }
48

```

Explain contract

AI copilot

Listen on all transactions

Filter with transaction hash or address

to: Counter.count() data: 0x066...61abd

call to Counter.count

call [call] from: 0x5838Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.count() data: 0x066...61abd

call to Counter.get

call [call] from: 0x5838Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x064...ce3c

- Tutorial no.16:

LEARNETH

Tutorials list

Syllabus

9. Data Locations

16 / 19

Assignments from *storage* to *local storage* also create references, not copies.

If we change the value of the local variable `myStruct` (line 17), the value of our state variable `myStructs` (line 10) changes as well.

Storage and memory/calldata

Assignments between *storage* and *memory* (or *calldata*) create independent copies, not references.

If we were to create a new struct `myMemStruct3` with the data location *memory* inside the `function f` (line 12) and assign it the value of `myStruct`, changes in `myMemStruct3` would not affect the values stored in the mapping `myStructs` (line 10).

As we said in the beginning, when creating contracts we have to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

★ Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myMemStruct2` with the data location *memory* inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location *memory* inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

enums.sol

enums_answer.sol

```

22
23 MyStruct memory myMemStruct3 = myStruct;
24 myMemStruct3.foo = 3;
25 return (myStruct, myMemStruct2, myMemStruct3);
26 }
27
28 function _f(
29     uint[] storage _arr,
30     mapping(uint => address) storage _map,
31     MyStruct storage _myStruct
32 ) internal {
33     // do something with storage variables
34 }
35
36 // You can return memory variables
37 //Soham Satpute
38 function g(uint[] memory _arr) public returns (uint[])
39 {
40     // do something with memory array
41     _arr[0] = 1;
42 }
43
44 function h(uint[] calldata _arr) external
45 {
46     // do something with calldata array
47     _arr[0] = 1;
48 }

```

Explain contract

0

Listen on all transactions

call to Counter.count

call [call] from: 0x5838Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter

call to Counter.get

call [call] from: 0x5838Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter

- Tutorial no. 17

LEARN ETHER

Tutorials list
Syllabus

10.1 Transactions - Ether and Wei
17 / 19

Ether (ETH) is a cryptocurrency. Ether is also used to pay fees for using the Ethereum network, like making transactions in the form of sending Ether to an address or interacting with an Ethereum application.

Ether Units

To specify a unit of *Ether*, we can add the suffixes `wei`, `gwei`, or `ether` to a literal number.

`wei`

Wei is the smallest subunit of *Ether*, named after the cryptographer [Wei Dai](#). *Ether* numbers without a suffix are treated as `wei` (line 7).

`gwei`

One `gwei` (giga-wei) is equal to 1,000,000,000 (10⁹) `wei`.

`ether`

One `ether` is equal to 1,000,000,000,000,000 (10¹⁸) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

★ Assignment

- Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
- Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10⁹.

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer
Show answer

Next

Well done! No errors.

Compile
anums_answer.sol
dataLocations.sol
dataLocations_answer.sol

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    //Soham Satpute
16    bool public isOneGwei = 1 gwei == 1e9;
17 }
```

✎ Explain contract

0
☐ Listen on all transactions
🔍

```

call to Counter.count

[call] from: 0x58380a6a701c56854dCfC803fC8B75f56beddC4 to: Counter.count() data: 0x066...

call to Counter.get

[call] from: 0x58380a6a701c56854dCfC803fC8B75f56beddC4 to: Counter.get() data: 0xd6d4...

```

- Tutorial no.18:

LEARNETH

← Tutorials list
≡ Syllabus

←
10.2 Transactions - Gas and Gas Price
→

18 / 19

consumes is mostly the same, but the price you pay for gas depends on the market.

Similarly, the amount of *gas* that a transaction requires is always the same for the same computational work that is associated with it. However the price that the sender of the transaction is willing to pay for the *gas* is up to them. Transactions with higher *gas* prices are going through faster; transactions with very low *gas* prices might not go through at all.

When sending a transaction, the sender has to pay the *gas* fee (*gas*_price * *gas*) upon execution of the transaction. If *gas* is left over after the execution is completed, the sender gets refunded.

Gas prices are denoted in gwei.

Gas limit

When sending a transaction, the sender specifies the maximum amount of *gas* that they are willing to pay for. If they set the limit too low, their transaction can run out of *gas* before being completed, reverting any changes being made. In this case, the *gas* was consumed and can't be refunded.

Learn more about *gas* on ethereum.org.

[Watch a video tutorial on Gas and Gas Price.](#)

★ Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the *gas* cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the *gas* cost. You can also use the Remix plugin *Gas Profiler* to check for the *gas* cost of transactions.

Check Answer
Show answer

Well done! No errors.

→ Compilers
→ etherAndWeiSol2
→ etherAndSol2

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4
5 contract Gas {
6     uint public i = 0;
7     uint public cost = 170367;
8
9     // Using up all of the gas that you send causes your tran
10    // State changes are undone.
11    // Gas spent are not refunded.
12    function forever() public { // infinite gas
13        // Here we run a loop until all of the gas are spent
14        // and the transaction fails
15        //Soham Satpute
16        while (true) {
17            i += 1;
18        }
19    }

```

✖ Explain contract

0
☐ Listen on all

call to Counter.count

call [call] from: 0x5380da701c568545dcfc803fc8875f56bedd4 to: Counter.cou

call to Counter.get

call [call] from: 0x5380da701c568545dcfc803fc8875f56bedd4 to: Counter.get

- Tutorial no. 19

LEARNETH

Tutorials list

Syllabus

10.3 Transactions - Sending Ether
19 / 19

and 45) also need to be `payable` in order to receive Ether.

Payable address

Solidity makes a distinction between two different flavors of the address data type: `address` and `address payable`.

`address`: Holds a 20-byte value. `address payable`: Holds a 20-byte value and can receive Ether via its members: `transfer` and `send`.

If you change the parameter type for the functions `sendViaTransfer` and `sendViaSend` (line 33 and 38) from `payable address` to `address`, you won't be able to use `transfer()` (line 35) or `send()` (line 41).

Watch a video tutorial on Sending Ether.

★ Assignment

Build a charity contract that receives Ether that can be withdrawn by a beneficiary.

1. Create a contract called `Charity`.
2. Add a public state variable called `owner` of the type `address`.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Check Answer

Show answer

Next

Well done! No errors.

Compile

jWei_Ansvers.sol gasAndGasPrice.sol gasAndGasPric

```
46 // call returns a boolean value indicating success or failure
47 // This is the current recommended method to use.
48 (bool sent, bytes memory data) = _to.call{value: msg.val
49     require(sent, "Failed to send Ether");
50 }
51 }
52 //Soham Satpute
53 contract Charity {
54     address public owner;
55
56     constructor() {
57         owner = msg.sender;
58     }
59
60     function donate() public payable {}
61
62     function withdraw() public {
63         uint amount = address(this).balance;
64
65         (bool sent, bytes memory data) = owner.call{value: amoun
66             require(sent, "Failed to send Ether");
67     }
68 }
```

Explain contract

0 ☐ Listen on all trans

call to Counter.count

call [call] from: 0x58380a6a701c568545dcfc803Fc8875f56beddC4 to: Counter.count()

call to Counter.get

call [call] from: 0x58380a6a701c568545dcfc803Fc8875f56beddC4 to: Counter.get() d

Conclusion: Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.